EPOCH

INDIVIDUAL WORK ALLOCATION & DELIVERY PLAN

Sushma Gouda | 30% of total team workload

Project	EPOCH — Research Integrity & Evidence Workspace
Assigned share	30%
Role	Backend Platform, Data Model & Workflow API Owner
Architecture baseline	EPOCH_Final_System_Architecture_v2.md (12 November 2026)
Document purpose	Individual execution brief — tasks, deliverables, dependencies and acceptance criteria

Mission

Build the trusted application layer that stores investigations and enforces who can upload data, author plans, approve runs, inspect results and export records.

Ownership boundary

Primary owner for FastAPI modular monolith, PostgreSQL schema/migrations, authentication/authorisation, investigation workflow state machines, plan approval/freeze endpoints, run ledger and audit-chain persistence. Coordinates engine calls but does not duplicate statistical logic.

Workload allocation

This assignment represents 30% of the planned team effort. The percentage is a workload share, not a claim that the work is already complete. Scope changes must be agreed by the team and recorded; do not silently transfer unfinished tasks.

Non-negotiable project rules

- Integrity Core must work end-to-end before optional extensions are treated as deliverables.
- Human researchers author hypotheses and plans; the system records, constrains, evaluates and audits the workflow.
- The sealed confirmation split must not be exposed to exploration, ordinary API responses, or AI features.
- Every confirmatory plan is approved and frozen before execution; each confirmatory plan consumes one unit of the lineage budget K.
- Post-hoc work is labelled and descriptive; it must never be promoted to Supported.
- AI is optional, off by default, and never performs statistical calculations or silently edits a human-owned plan.
- All results must be reproducible and verifiable from a signed evidence bundle.

1. Detailed task breakdown

M1 — Backend foundation and schema

- Set up FastAPI, Pydantic, SQLAlchemy and Alembic structure. Organise modules: auth/authz, investigations, data, hypotheses, plans, runs, evidence/graph, reports, audit and ai-adapter.
- Design the single relational schema from v2 §9: investigation/membership, dataset/version/lineage, hypothesis/prediction, plan/version/approval, run/result, evidence/claim links, audit events and bundle metadata.
- Add database constraints, foreign keys, indexes and migrations. Document ownership and lifecycle of each record; avoid duplicate sources of truth.

M2 — Identity, access control and investigation workflow

- Implement sign-in using a managed provider or approved library; use secure HttpOnly/Secure/SameSite cookies and CSRF protection where applicable.
- Implement owner and reviewer membership. Every object route must resolve object → investigation → membership; deny non-members with 403/404.
- Implement investigation creation, question/owner/reviewer metadata, data-version registration, hypothesis CRUD and immutable revision history.
- Implement explicit state transitions for draft, submitted, approved, frozen, queued, running, completed, failed and post-hoc records; reject illegal transitions server-side.

M3 — Plan, approval, budget and audit

- Implement plan submission and reviewer approval. If two-person approval is enabled, author and approver must differ.
- On freeze, persist plan_hash, split-membership commitment, engine version, budget state and mandatory external RFC 3161 timestamp token/reference before a run can start.
- Enforce budget K at first plan submission/lock. Each confirmatory plan consumes one unit under v2 rules; ensure retries and duplicate workers cannot double-spend.
- Record every attempt in an append-only run ledger. Use INSERT-only app role for audit_event; reject UPDATE/DELETE. Maintain hash-chain fields and checkpoint verification references.
- Implement amendment workflow with reviewer approval and explicit out-of-FWER status; amended runs cannot be Supported except permitted replication on new rows.

M4 — API, reports and bundle endpoints

- Expose versioned endpoints for investigations, data versions, hypotheses, plans, approvals, runs, evidence, report and bundle download.
- Validate every request/response with Pydantic models; do not leak confirmation rows, secret salt, raw executor paths or credentials.
- Implement report assembly from stored records and safe engine outputs; include operator, limitations, plan/run lineage, audit head, provenance and post-hoc/amendment labels.
- Use content-addressed storage references for uploads, canonical data, artifacts and signed bundles; signed URLs must expire within five minutes or less as stated in v2.
- Provide OpenAPI docs and sample requests/responses for Pranav and Atul.

2. Required deliverables

Every item below must be produced, committed or linked, and demonstrated. A verbal update alone is not a deliverable.

- Runnable FastAPI service with modular route layout and configuration.
- PostgreSQL schema, Alembic migrations, ER/data dictionary and lifecycle/state-machine documentation.
- Authentication and investigation-level authorisation with route-level tests.
- Plan submission/approval/freeze APIs, budget K enforcement, audit chain and timestamp integration.
- Run ledger and report/bundle API integration with content-addressed storage.
- OpenAPI contract, sample payloads, local setup instructions and backend handoff.

3. Definition of done and acceptance criteria

A deliverable is accepted only when the relevant evidence exists (tests, API contract, migration, screenshot plus working route, reproducible command, or review record).

- All investigation objects are inaccessible to non-members (403/404); route tests cover every module.
- Illegal plan/run state transitions are rejected; a confirmatory run requires approved and frozen plan.
- Two-person approval rejects the author approving their own plan when enabled.
- Duplicate jobs/retries cannot double-spend K; every attempt remains visible in the ledger.
- Audit UPDATE/DELETE is rejected for the application role; tampered chain/checkpoint fails verification.
- Secret split salt and confirmation rows never appear in API responses or logs; signed URLs expire within the configured limit.

4. Dependencies and handoffs

- Adithya supplies the engine public API, split/freeze commitment and verdict schemas.
- Atul supplies the executor job/result contract, supervisor lifecycle and failure semantics.
- Pranav supplies UI requirements and consumes the OpenAPI contract; coordinate error messages and workflow states.
- Team must decide authentication provider, external timestamp authority, storage location and deployment environment.

5. Explicitly out of scope

- Do not calculate statistical tests or verdicts inside FastAPI; call epoch-engine.
- Do not let user-uploaded code execute in API or supervisor.
- Do not build multi-organisation RBAC, real-time collaboration, arbitrary code execution or graph database in MVP.

6. Milestones and sequencing

Follow the architecture's M0–M7 build order. The dates are intentionally not invented here; the team should set calendar dates after M0 confirms the dataset, question, capacity and technical feasibility.

Milestone	Your expected contribution
M0 — Decide	Confirm entity ownership, user roles, auth choice, data handling and K policy.
M1 — Engine/API contract	Create schema and API skeleton; mock engine outputs until engine contract is stable.
M2 — Backend	Complete auth, investigation/data/hypothesis/plan/run records, queue integration and audit.
M3 — UI slice	Support the complete vertical slice with Pranav's screens and Atul's executor.
M4–M6 — Extensions/hardening	Add E1/E2/E3 data surfaces only after Core; support benchmark and red-team evidence.

7. Required weekly progress update

- Completed: list merged work and link the PR/commit, migration, test report or document.
- In progress: identify the exact task, current status and next concrete action.
- Blocked: name the dependency, who can unblock it and the decision needed.
- Evidence: provide command output, test result, API response, short recording or reproducible demo.
- Risk/change: state any scope change, security concern, schema change or estimate change before acting on it.

8. Collaboration and handoff checklist

- Agree interface/schema changes with dependent owners before implementation; avoid parallel incompatible assumptions.
- Open a focused pull request with a clear summary, test evidence and known limitations.
- Include setup/run instructions and sample configuration without secrets.
- Never commit credentials, signing keys, dataset secrets or personal/regulated data.
- Demonstrate the feature from a clean checkout or documented environment before marking it complete.

9. Review 1 contribution

For Review 1, present the data model, API module map, approval/freeze/budget state machine, authorisation model and how every run becomes an immutable auditable record. Label timestamping and security controls as planned until tested.

10. Shared project completion standard

- All code is committed to the agreed repository and the team can run it from documented setup instructions.
- Interfaces, request/response schemas, database migrations and environment variables are documented.
- Unit tests and integration tests are included; no task is considered complete on the basis of a screenshot alone.
- Known limitations, assumptions, risks and deferred features are recorded rather than hidden.
- Changes are reviewed by the relevant module owner and integrated through the shared branch/PR process.

11. Architecture traceability

This work plan is derived from the consolidated v2 architecture. Primary sections relevant to this assignment: §3 Scope tiers; §4 System architecture; §6 Website journey; §7 Integrity Core; §8 Extensions; §9 Data model; §10 Engine interface and test catalogue; §11 AI guardrails; §12 Security/privacy/reliability; §13 Evaluation/benchmark; §14 Build order; §15 Acceptance criteria; §16 Risks; §17 Open decisions; §21 v2 corrections. The individual plan does not override the architecture.

Important status note: the architecture is a proposed design, not evidence of implementation, load testing or security certification. Each owner must report actual implementation status honestly.